

Towards Fully Anonymous Integrity Checking and Reliability Authentication for Cloud Data Sharing

Ran Ding , Yan Xu , Hong Zhong , *Member, IEEE*, Jie Cui , and Kewei Sha , *Senior Member, IEEE*

Abstract—Cloud storage services improve the efficiency and popularity of data sharing. These services allow groups of participants to jointly maintain the shared data. As important security properties of cloud data sharing, the integrity and the reliability of the shared data have been studied recently. However, the existing research cannot sufficiently solve the issue of participant identity anonymity in the scenario of data modification. In this paper, we propose a novel approach to achieve fully anonymous integrity checking and reliability authentication for cloud data sharing. We design a predicate for the shared data, and construct the Lagrange interpolation polynomials for all participants to compute the secret keys based on the designed predicate. When modifying the shared data, the participants compute the authenticators of the modified data using the secret key associated with the designed predicate instead of their identities. In this way, the integrity checking and reliability authentication of the shared data modified by different participants can be performed while the identities of the corresponding participants remain fully anonymous. In addition, the traceability and revocation of participant identity are considered. The performance analysis demonstrates the efficiency of the proposed approach, and the security analysis shows that the proposed approach satisfies the desired properties.

Index Terms—Cloud data sharing, full identity anonymity, integrity checking, reliability authentication.

I. INTRODUCTION

CLOUD storage services, such as Google Cloud, Dropbox, Amazon EC2, and FusionCloud, have gained popularity

Manuscript received 26 June 2022; revised 14 March 2023; accepted 23 April 2023. Date of publication 26 April 2023; date of current version 8 October 2023. The work was supported in part by the National Natural Science Foundation of China under Grants U1936220 and 61702005, in part by the Special Fund for Key Program of Science and Technology of Anhui Province, China under Grant 18030901027 and Anhui Provincial Natural Science Foundation under Grant 2008085QF297. Recommended for acceptance by H. Karatza. (*Corresponding author: Hong Zhong.*)

Ran Ding is with the School of Biomedical Engineering, Anhui Medical University, Hefei 230032, China, also with the Center for Medical Data Analysis and Supercomputing, Anhui Medical University, Hefei 230032, China, also with the Institutes of Physical Science and Information Technology, Anhui University, Hefei 230039, China, also with the School of Computer Science and Technology, Anhui University, Hefei 230039, China, and also with the Anhui Engineering Laboratory of IoT Security Technologies, Anhui University, Hefei 230039, China (e-mail: randing99@163.com).

Yan Xu, Hong Zhong, and Jie Cui are with the School of Computer Science and Technology, Anhui University, Hefei 230039, China, and also with the Anhui Engineering Laboratory of IoT Security Technologies, Anhui University, Hefei 230039, China (e-mail: xuyan@ahu.edu.cn; zhongh@ahu.edu.cn; cuijie@mail.ustc.edu.cn).

Kewei Sha is with the Department of Computer Science, University of Houston-Clear Lake, Houston, TX 77058 USA (e-mail: comersha@gmail.com). This article has supplementary downloadable material available at <https://doi.org/10.1109/TSC.2023.3270580>, provided by the authors.

Digital Object Identifier 10.1109/TSC.2023.3270580

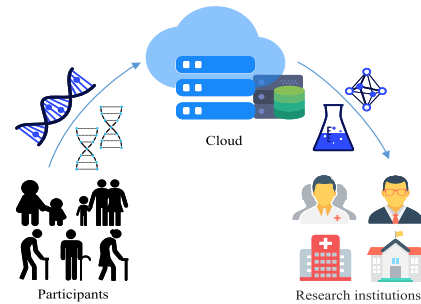


Fig. 1. Genomic data sharing based on the cloud server.

recently. The data sharing based on cloud storage provides a convenient way for a group of participants to share and modify data. After uploading the shared data to the cloud server for storage, the participants can delete the local copy to save storage resources, and access and modify the shared data through the cloud server. However, when hackers attack the cloud server or internal workers make mistakes, the data stored on the cloud server may be lost or tampered. In order to maintain its business reputation, the cloud server may hide such data corruption events. This is harmful to participants because they may be misled by the corrupted data without knowing it when accessing the shared data through the cloud server. For instance, cloud storage services can be leveraged to facilitate Genome-Wide Association Studies (GWAS), which aim to discover potential correlations between a disease and a certain gene [1], [2]. The participants upload genomic data to the cloud server and share their genomes to a study [3], [4], as shown in Fig. 1. Research institutions access and analyze the shared genomic data through the cloud server. But they may get incomplete even wrong correlations if the cloud server has lost part of the stored genomic data. Therefore, it is necessary to check the integrity of the data stored on the cloud server periodically. In addition, in order to prevent illegal participants from sharing data, it is necessary to ensure the reliability of data sources, i.e., reliability authentication. Existing work has investigated the integrity and the reliability of the shared data stored on the cloud server [5], [6], [7], [8]. In these studies, reliability authentication is performed by a group manager. Specifically, the group manager enrolls new participants and makes them legitimate by distributing secret keys. For the purpose of integrity checking, the legitimate participants then use their secret keys to generate valid authenticators of the data to be shared. After that, they upload the shared data and corresponding authenticators

to the cloud server. A third-party auditor (TPA) can request the cloud server to compute the storage proof of the shared data by using corresponding authenticators. Thereupon, the TPA executes the integrity checking by verifying the storage proof without downloading the shared data because the cloud server cannot generate valid authenticators for the corrupted data.

Generally, to a certain extent, the shared data may contain some sensitive information to maintain its utility. As described in the above example, in order to accurately discover potential correlations, the sensitive information about a participant, such as personal traits, health problems, predispositions to diseases, familial relationships, etc., will be included in his genomic data. In this case, if the identity of the uploader corresponding to the shared data is known by any party, i.e., the TPA, cloud server, group manager, or other participants, such as research institutions in the group, the uploader may be troubled. For instance, it can be inferred from the genomic data uploaded by a participant that he carries the mutation of a specific gene which increases the likelihood of developing a specific disease. Then the insurance company may record his identity to deny his health coverage [9], [10], [11]. As a result, more and more participants will be reluctant to share their genomic data. Thus, it is critical to hide the participants' identities in such applications, i.e., achieving full identity anonymity. What's more, some government departments have concerned the privacy disclosure in data sharing and promulgated some regulations, such as the EU General Data Protection Regulation (GDPR) in 2018. Thus, protecting personal privacy is the general trend. Specifically, for cloud data sharing, it is essential to achieve full identity anonymity. Unfortunately, in the work described above [5], [6], [7], [8], a participant identity is known by at least one of the TPA, cloud server, group manager, and other participants in the group. The work in [5] proposes a homomorphic authenticable ring signature to hide the relationship between the signature and the corresponding participant. Nevertheless, the participant identity in [5] is known by the TPA, cloud server, group manager, and other participants in the group. Although Hu et al. designed an integrity checking approach where the participant identity is anonymous to the cloud server [6], the group manager can reveal the participant identity [6]. Similarly, the participant identity is anonymous to the TPA in [7], but both the cloud server and the group manager can find out the participant identity. Although the participant identity is anonymous to the cloud server and TPA in [8], it is known by the group manager and other group participants. So far, no existing work solves the issue of full identity anonymity, particularly identity anonymity within the group.

Yet, as far as we know, it is difficult to achieve full identity anonymity in existing work, which is caused by the data modification. Specifically, the data modification performed by different participants, as an indispensable function of cloud data sharing, is supported in [5], [6], [7], [8]. However, as mentioned above, the participants have to use their secret keys to compute the authenticators for the data uploaded during data modification [5], [6], [7], [8]. In order to prove that the data is well stored, the cloud server aggregates several stored data blocks and corresponding authenticators to obtain storage proof and sends to the TPA. It

should be noted that the TPA must use the public keys of the participants who compute the authenticators when checking the integrity and the reliability of the shared data by verifying the validity of the storage proof. As a result, the participant identities cannot be completely protected in cloud data sharing [5], [6], [7], [8]. Moreover, the data modification and authenticator generation are performed at the block level. Each participant in the group may modify multiple data blocks. In integrity checking, the TPA randomly selects a certain number of data blocks for checking, and the data blocks contained therein may be modified by several different participants. It is challenging for TPA to check the integrity of the data at one time while keeping the participant identities fully anonymous. A potential solution is to issue a common secret key for all participants to compute the authenticators. However, the use of the common secret key makes it difficult to verify the legitimacy of the participant that uploads the shared data, i.e., the reliability authentication is infeasible. Since cloud data sharing is a complex environment, in order to ensure the security and credibility of data sharing, the cloud server and data users must be able to verify the reliability of data sources. Therefore, reliability authentication must be supported. Therefore, it is a challenge to support the integrity checking and reliability authentication of the modified data on the premise of maintaining full identity anonymity.

In order to address the above problems, we design a fully anonymous integrity checking and reliability authentication scheme by using the technique of threshold attribute-based signature (ABS) [12]. Although ABS can be used to support identity anonymity, it is infeasible to directly use it to construct an integrity checking scheme. There are two challenges. First, the participants' attribute sets are different. It is still challenging to check the integrity of different data blocks while keeping the participants' attribute sets anonymous. In addition, if all participants use the same attribute set to compute the authenticators, such as the attribute set contained in the threshold ABS, not all participants have the secret keys of the attributes in the set. Therefore, this paper is dedicated to addressing these challenges. The contributions of this paper can be summarized as follows:

- We propose a novel approach to achieve fully anonymous integrity checking and reliability authentication for cloud data sharing (SEAYS). Different from the existing schemes, SEAYS supports data modification while keeping the participant identity anonymous to the cloud server, TPA, and other participants in the group. Further, we consider the identity traceability and revocation without compromising identity anonymity.
- We introduce a designed predicate, and associate the shared data and the corresponding authenticators with it rather than the participant's identity. In this way, the participant identity remains anonymous to the cloud server. Moreover, the participant identity remains anonymous to the TPA, because the TPA only needs the designed predicate instead of the participant identities when checking the integrity of the shared data.
- We design a novel authenticator generation method. To be specific, we improve the original ABS scheme to compute the authenticators of the data modified by different

participants. To this end, we construct two Lagrange interpolation polynomials for each participant based on the designed predicate to distribute the secret key. A participant uses his own secret key to compute the authenticators of the modified data without communicating with others, making his identity remain anonymous to other participants. Furthermore, the reliability of the shared data is guaranteed, because only the legitimate participant has the secret keys to compute the valid authenticators.

- We conduct comprehensive simulation experiments for SEAYS. The results show that SEAYS achieves full identity anonymity while maintaining reasonable computation cost. Moreover, we prove the security of SEAYS under the random oracle model.

The remainder of this paper is organized as follows. Section II describes the preliminaries used in SEAYS. Section III introduces the system model, problem formulation, algorithm definition. Section IV demonstrates the detailed construction of SEAYS. Section V analyzes the performance of SEAYS. Section VI reviews the work related to this study. Section VII concludes the paper. Due to page limitation, part of the paper has been moved to the appendix. The background knowledge is described in Appendix A, (available online). Appendix B, available in the online supplemental material introduces the security model of SEAYS. Appendix C, available in the online supplemental material analyzes the performance of SEAYS numerically. The correctness analysis and security proof of SEAYS are introduced in Appendix D, available in the online supplemental material.

II. PRELIMINARIES

This section introduces the background knowledge used in SEAYS, including the Lagrange interpolation [13] and threshold ABS [14]. The bilinear pairing [15], Discrete Logarithm (DL) assumption [16], Computational Diffie-Hellman (CDH) assumption [17], and Bilinear Diffie-Hellman (BDH) assumption [18] will be introduced in Appendix A, available in the online supplemental material.

A. Lagrange Interpolation

Let $q(x)$ be a $d - 1$ degree polynomial, and S be the set of d different points $q(1), q(2), \dots, q(d)$ of polynomial $q(x)$. For $\forall i \in Z_p$, we can compute the polynomial value $q(i)$ with Lagrange interpolation. We define the Lagrange coefficient $\Delta_{j,S}(i)$ of point $q(j)_{1 \leq j \leq d}$ as $\Delta_{j,S}(i) = \prod_{k \in S, k \neq j} \frac{i-k}{j-k}$. The polynomial value $q(i)$ can be computed as

$$q(i) = \sum_{j=1}^d q(j) \cdot \Delta_{j,S}(i).$$

B. Threshold Attribute-Based Signature

Assume the attribute universe of the threshold ABS is $\mathbb{U}$, and a signer's attribute set ω satisfies $\omega \subseteq \mathbb{U}$. In a threshold ABS scheme, the authority generates the secret key for a signer according to ω , and the signer utilizes the secret key to generate

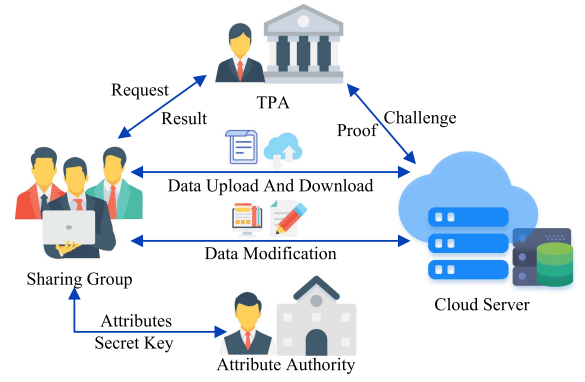


Fig. 2. The system model of SEAYS.

a signature of the message m under a predicate $\Upsilon_{d,S}$, where $S \subseteq \mathbb{U}$ is the set of attributes included in the predicate and d is the threshold that satisfies $1 \leq d \leq |S|$. We say the signer's attribute set ω satisfies the predicate $\Upsilon_{d,S}$ if $|\omega \cap S| \geq d$. The signature will only pass verification if the signer's attribute set ω satisfies $\Upsilon_{d,S}$. The concrete procedure of threshold ABS is displayed as follows.

- 1) *Setup*($1^\lambda, \mathbb{U}$): With the security parameter λ and the attribute universe $\mathbb{U}$, the authority outputs the master private key msk and the master public key mpk .
- 2) *KeyGen*(msk, ω): With the master private key msk , the authority computes the signer's secret key SK_ω according to attribute set ω .
- 3) *Signing*($mpk, SK_\omega, m, \Upsilon_{d,S}$): A signer whose attribute set satisfies the predicate $\Upsilon_{d,S}$ computes the signature σ of the message m with his secret key SK_ω .
- 4) *Verifying*($mpk, \Upsilon_{d,S}, m, \sigma$): With the master public key mpk , predicate $\Upsilon_{d,S}$, message m , and signature σ , this algorithm verifies the validity of the signature σ .

III. SYSTEM DESCRIPTION AND PROBLEM FORMULATION

This section describes the system model, problem formulation, and the algorithm definition.

A. System Model

As shown in Fig. 2, the system of SEAYS involves the sharing group, attribute authority, cloud server, and TPA. The following is a detailed description.

Sharing Group: Participants whose attributes satisfy the designed predicate form the sharing group. No group manager is needed in the sharing group. Any participant in the sharing group can share and modify the data stored on the cloud server. Participants in the group remain anonymous to each other.

Attribute Authority: The attribute authority verifies the participant's attributes and issues a secret key for the participant according to the corresponding attributes. We assume that the attribute authority is trusted in SEAYS.

Cloud Server: The cloud server provides storage services for the sharing group and updates the stored data after receiving the modification request from the participants. To avoid disputes, the

cloud server can request the TPA to verify the integrity and the reliability of the data uploaded by participants before storing the data. We assume that the cloud server performs the algorithms honestly but is curious about the participant identities and may hide the data loss events.

TPA: The TPA can be the participant or a trusted third auditor and executes the integrity checking and reliability authentication for the shared data. We assume that the TPA performs the algorithms honestly but is curious about the participant identities.

B. Problem Formulation

In this subsection, we describe the problem studied in this paper by introducing the definition of SEAYS and the corresponding security requirements.

Definition 1 (Fully Anonymous Integrity Checking And Reliability Authentication). Given the data m that is shared by a group of participants through the cloud storage service, the goal of SEAYS is to check whether the cloud server loses the stored data m and authenticate whether m is uploaded by legitimate participants, i.e., checking the integrity and authenticating the reliability of m .

For the above definition, we need to consider the following five security requirements.

First, the integrity checking and reliability authentication needs to be performed without downloading the whole data to reduce communication overhead. Thus, the TPA needs the cloud server to send a storage proof to verify whether the data is well stored on the cloud server. Therefore, the first security requirement of SEAYS is *Unforgeability*. *Unforgeability* indicates that the data modified by all legitimate participants is well stored on the cloud server if the TPA outputs “True” for the cloud server’s proof. In other words, the broken or forged data cannot pass the integrity checking and reliability authentication.

Second, SEAYS ensures that the data is shared by the legitimate participants through reliability authentication. Therefore, SEAYS needs to satisfy the security requirement of *Collusion Resilience*. That is to say, none of the participants whose attribute sets do not satisfy the predicate can generate valid authenticators and join the group even if they cooperate with each other.

Third, the participants are concerned about their identity privacy, so SEAYS needs to satisfy the security requirement of *Full Identity Anonymity*. During data sharing, the participants in the group can modify the data and generate the corresponding authenticators without knowing each other’s identity. Besides, the cloud server stores the data uploaded by the participant without knowing his identity. Furthermore, the TPA executes the integrity checking and reliability authentication of the data modified by different participants without knowing their identities.

In addition, when the participant in the group performs malicious data modification or other bad behaviors, SEAYS should be able to track the participant identity, i.e., the security requirement of identity traceability.

Finally, when there is malicious participant in the group, or the participant requests to exit data sharing, SEAYS should be able

to revoke the participant identity, that is, the security requirement of identity revocation.

C. Algorithm Definition

The system of SEAYS includes three modules *Initialization*, *Sharing*, and *Verification*. First, we introduce the algorithm definition of the *Initialization* module. In this module, the authority generates system parameters and secret keys.

- 1) *Setup*(1^λ): The authority executes the *Setup* algorithm. With the security parameter λ , the authority computes the master private key msk and the master public key mpk of the system. Additionally, the authority specifies the predicate $\mathcal{T}_{d,s}$ for the shared data and publishes it along with mpk .
- 2) *KeyGen*($msk, mpk, \omega_u, \mathbb{U}$): The authority executes the *KeyGen* algorithm. With msk , mpk , and $\mathcal{T}_{d,s}$, the authority generates the secret key SK_u of the participant u according to his attribute set $\omega_u \subseteq \mathbb{U}$ and distributes the secret key to the participant u through a secure channel between the authority and u .

In the *Sharing* module, participants generate the shared data’s authenticators used in the *Verification* module and modify the shared data with the cloud server’s help. In this module, the shared data m is divided into a series of data blocks. Thus, the data modification and authenticator generation are performed at the block level. The algorithm definition of this module is as follows.

- 1) *AuthGen*(ω_u, m, mpk, SK_u): A participant executes the *AuthGen* algorithm. With mpk , his attribute set ω_u , and secret key SK_u , the participant computes the authenticators σ for data m using the specified predicate $\mathcal{T}_{d,s}$.
- 2) *Modification*(ω_u, Com, mpk, SK_u): With mpk , his attribute set ω_u , and the secret key SK_u , a participant generates the modified data block and computes new authenticators for the modified data block. Then, the participant generates a modification command Com that includes the modified data, new authenticators, and modification type. After that, the participant sends Com to the cloud server. The cloud server updates the stored data and authenticators according to Com .

In the *Verification* module, the TPA checks the integrity of the shared data and authenticates the reliability of the shared data by executing the challenge-proof protocol with the cloud server. The algorithm definition of the *Verification* module is as follows.

- 1) *ChalGen*(mpk): The TPA executes the *ChalGen* algorithm. After receiving the integrity checking and reliability authentication request issued by a participant, the TPA generates a challenge $Chal$ that indicates the checked data and sends it to the cloud server.
- 2) *ProofGen*($Chal, \sigma, m$): The cloud server aggregates the stored data and the corresponding authenticators with the parameters specified in $Chal$ to compute the storage proof Prf . After that, the cloud server sends Prf to the TPA.

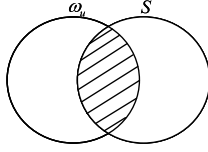


Fig. 3. The participant's attribute set ω_u and the attribute set S contained in the predicate $\mathcal{T}_{d,S}$.

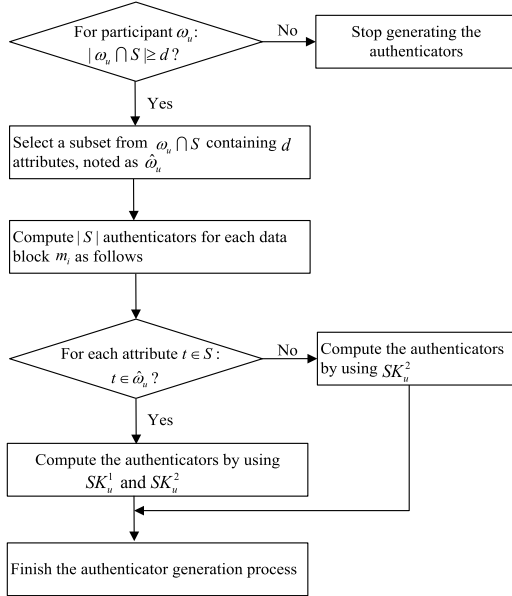


Fig. 4. The process flow diagram of the method of authenticator generation.

- 3) *CheckCert*($mpk, Chal, Prf$): With mpk and $\mathcal{T}_{d,S}$, the TPA performs the integrity checking and reliability authentication for the shared data by verifying the validity of Prf . If Prf is valid, the TPA outputs “True”, which indicates that the shared data are intact and shared by legitimate participants in the group. Otherwise, the TPA outputs “False”.

IV. OUR CONSTRUCTION

In this section, we introduce the main idea of SEAYS, the technique to support data modification, the construction of SEAYS, and the identity verification of SEAYS.

A. The Main Idea of SEAYS

In SEAYS, the identity of a participant u is associated with an attribute set ω_u , and predicate $\mathcal{T}_{d,S}$ contains an attribute set S , so the group consists of participants whose attribute sets satisfy the predicate, i.e., $|\omega_u \cap S| \geq d$, as shown in Fig. 3.

First, the authority generates two Lagrange polynomials for each legitimate participant based on the master secret key and computes the participant's secret key with the polynomials. Specifically, part of the secret key SK_u^2 is computed for all attributes in S , and another part SK_u^1 is computed for the participant's attribute set ω_u to judge whether the participant is legitimate. As shown in Fig. 4, each legitimate participant can generate valid authenticators for the data by using his own secret

key. Based on the property of Lagrange interpolation, the TPA can use the public parameters to perform the integrity checking and reliability authentication of data modified by different participants without knowing their identities.

In SEAYS, each participant computes the authenticators for all attributes in the predicate, i.e., the attributes in S , even if he does not have all the attributes. To this end, we divide the attributes in S into two subsets in the authenticator generation algorithm, as shown in Fig. 4. One subset contains the attributes that a participant uses to satisfy the predicate, i.e., the attribute set $\hat{\omega}_u$, and the remaining attributes in S constitute the other subset S_u . The participant divides the data m into multiple data blocks. For each data block m_i , the participant computes $|S|$ authenticators corresponding to each attribute in S . Since the participant does not have all attributes in S , the methods to compute the authenticators of m_i are different for attributes in different subsets. Specifically, the participant can use SK_u^1 and SK_u^2 to compute authenticators for attributes in $\hat{\omega}_u$. For the attributes in S_u , the participant can compute the corresponding authenticators only by using SK_u^2 . In this way, the information about the participant's identity will not be revealed from the authenticators.

Therefore, SEAYS supports the integrity checking and reliability authentication of the data while keeping the participant's identity anonymous to the cloud server, TPA, and other participants in the group.

In order to illustrate the ideas of SEAYS more clearly, we give the flow chart of the scheme, as shown in Fig. 5.

B. Index-Hash Table

In SEAYS, the shared data m is initially divided into n data blocks $\{m_i\}_{1 \leq i \leq n}$, and each data block m_i is further divided into s sectors $\{m_{ij}\}_{1 \leq j \leq s}$. The participants perform the modification on data blocks. We utilize the index-hash table [5] to support efficient data modification and deal with index changes.

As shown in Fig. 6(a), the initial data owner constructs the original index-hash table of the shared data before uploading them to the cloud server. The cloud server maintains the index-hash table. There are four items in the index-hash table. The index item indicates the serial number of the data blocks, and differs from each other. The data block item stores the actual block. The virtual number item $\mathcal{V}$ is used to remain the right order of the data blocks. For example, when inserting the data block m' before the data block m_i , the virtual number of m' is $\frac{\mathcal{V}_i + \mathcal{V}_{i-1}}{2}$. The parameter θ is selected by the participant constructing the original index-hash table. The max number of blocks that can be inserted between two adjacent blocks is θ . The random number item $\mathcal{R}$ is a value computed by a collision-resistance hash function H_2 using the data block and corresponding virtual number. For instance, the random number $\mathcal{R}_i$ of the data block m_i is $H_2(\sum_{j=1}^s m_{ij} || \mathcal{V}_i)$.

The identifier of m_i is uniquely represented by $(\mathcal{V}_i, \mathcal{R}_i)$. Fig. 6(b) shows the index-hash table after inserting the data block m'_2 at index 2. The index-hash table after updating block m_1 and deleting block m_2 is shown in Fig. 6(c).

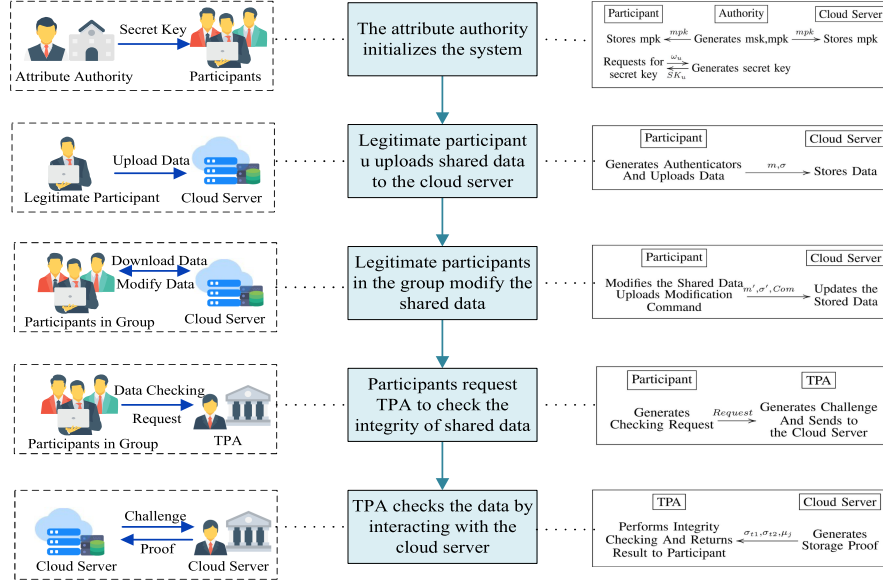


Fig. 5. The flow chart of SEAYS.

Index	Data Block	Virtual Number	Random Number
1	m_1	θ	r_1
2	m_2	$2 \cdot \theta$	r_2
3	m_3	$3 \cdot \theta$	r_3
4	m_4	$4 \cdot \theta$	r_4
$\vdots$	$\vdots$	$\vdots$	$\vdots$
$n-1$	m_{n-1}	$(n-1) \cdot \theta$	r_{n-1}
n	m_n	$n \cdot \theta$	r_n

(a) The original index-hash table

Index	Data Block	Virtual Number	Random Number
1	m_1	θ	r_1
2	m'_2	$\frac{3}{2} \cdot \theta$	r'_2
3	m_2	$2 \cdot \theta$	r_2
4	m_3	$3 \cdot \theta$	r_3
$\vdots$	$\vdots$	$\vdots$	$\vdots$
n	m_{n-1}	$(n-1) \cdot \theta$	r_{n-1}
$n+1$	m_n	$n \cdot \theta$	r_n

(b) After inserting a block

Index	Data Block	Virtual Number	Random Number
1	m'_1	θ	r'_1
2	m_3	$3 \cdot \theta$	r_3
3	m_4	$4 \cdot \theta$	r_4
4	m_5	$5 \cdot \theta$	r_5
$\vdots$	$\vdots$	$\vdots$	$\vdots$
$n-2$	m_{n-1}	$(n-1) \cdot \theta$	r_{n-1}
$n-1$	m_n	$n \cdot \theta$	r_n

(c) After deleting a block

Fig. 6. The index-hash table of the shared data.

C. Construction of SEAYS

The detailed construction of SEAYS is as follows. In the construction, we denote the attribute universe as $\mathbb{U}$ and every attribute as $t \in Z_p$.

First, we describe the construction of the *Initialization* module in detail. The *Initialization* module includes the algorithms of *Setup* and *KeyGen*, which respectively handle the initialization of the system and the joining of new participants.

- 1) *Setup*(1^λ): The authority chooses two multiplicative cyclic groups $\mathbb{G}_1$ and $\mathbb{G}_T$ with the prime order p , and the bilinear pairing $e : \mathbb{G}_1 \times \mathbb{G}_1 \rightarrow \mathbb{G}_T$. Let hash functions $H_1 : \{0, 1\}^* \rightarrow \mathbb{G}_1$, $H_2 : \{0, 1\}^* \rightarrow Z_p^*$, and $H_3 : \{0, 1\}^* \rightarrow \mathbb{G}_1$. The authority randomly chooses generators $g, g_2 \in \mathbb{G}_1$, and numbers $x, y \in Z_p^*$, and computes $g_1 = g^x$, $Y = g^y$. The authority specifies the predicate $\Upsilon_{d,S}$ and a standard attribute-based signature scheme *ABS* with the private key *skey* and the public parameter *pkey* for the cloud data sharing. The authority sets the master private key $msk = (x, y)$ and the public parameters $mpk = (g, g_1, g_2, Y, H_1, H_2, H_3, e, \Upsilon_{d,S}, pkey)$.
- 2) *KeyGen*($msk, mpk, \omega_u, \mathbb{U}$): For a participant u , the authority computes the secret key SK_u according to his attribute set $\omega_u \subseteq \mathbb{U}$. The authority randomly chooses a $d-1$ degree polynomial q_u that satisfies $q_u(0) = x$ and a $|S|-1$ degree polynomial $\hat{q}_u$ that satisfies $\hat{q}_u(0) = y$.

For each attribute $t \in \omega_u$, the authority randomly chooses $r_t \in Z_p^*$ and computes $SK_{t1}^u = g_2^{q_u(t)} \cdot H_1(t)^{r_t}$, $SK_{t2}^u = g^{r_t}$. The authority sets $SK_u^u = \{SK_{t1}^u, SK_{t2}^u\}_{t \in \omega_u}$. If the attribute set ω_u does not satisfy the predicate $\Upsilon_{d,S}$, the authority sends the secret key $SK_u = SK_u^u$ to u . Otherwise, the authority executes the following computation. For each attribute t in S , the authority computes $\hat{q}_u(t)$, and sets $SK_u^2 = \{\hat{q}_u(t)\}_{t \in S}$. Then, the authority sends the secret key $SK_u = (SK_u^1, SK_u^2)$ and the private key *skey* to the participant u .

By owning the secret key SK_u^1 and SK_u^2 , the participant can legally access and modify the sharing data, that is, becomes a legitimate participant in the sharing group. Therefore, the *KeyGen* algorithm can flexibly support the joining of new participants.

The *Sharing* module includes the algorithms of *AuthGen* and *Modification*. In this module, the participants share the data with a unique identifier *name* that is stored on the cloud server.

- 1) *AuthGen*(ω_u, m, mpk, SK_u): If $|\omega_u \cap S| \geq d$, the participant u can generate the authenticators for the data m under the designed predicate $\Upsilon_{d,S}$. The participant u selects a subset $\hat{\omega}_u \subseteq \omega_u \cap S$ that satisfies $|\hat{\omega}_u| = d$. Denote $S_u = \{t | t \in S, t \notin \hat{\omega}_u\}$. After that, u randomly chooses a $d-1$ degree polynomial q'_u that satisfies $q'_u(0) = 0$ and a $|S|-1$ degree polynomial $\hat{q}'_u$ that satisfies $\hat{q}'_u(0) = 0$. In order to

compute the authenticators, u divides the uploaded data m into n blocks and each block m_i into s sectors, i.e., $m = \{m_{ij}\}_{1 \leq i \leq n, 1 \leq j \leq s}$ and $m_{ij} \in \mathbb{Z}_p^*$. Then, u randomly chooses $\{u_j\}_{1 \leq j \leq s}$ from $\mathbb{G}_1$. For each data block m_i , the participant computes $\mathcal{R}_i = H_2(\sum_{j=1}^s m_{ij} || \mathcal{V}_i)$ and computes the authenticators of m_i as follows. For each attribute t in S , the participant randomly chooses $r'_t \in \mathbb{Z}_p^*$ and executes the following algorithm:

If $t \in \hat{w}_u$, the participant computes $q'_u(t)$, $\hat{q}'_u(t)$, $\mathcal{H}_i = H_3(\text{name} || \mathcal{V}_i || \mathcal{R}_i)$, and

$$\sigma_{it1} = (SK_{t1}^u)^{\frac{1}{\Delta_{t,S_u(0)}}} \cdot H_1(t)^{r'_t} \cdot g_2^{\frac{q'_u(t)}{\Delta_{t,S_u(0)}}}.$$

$$\left(\prod_{j=1}^s u_j^{m_{ij}} \cdot \mathcal{H}_i \right)^{\hat{q}_u(t) + \hat{q}'_u(t)}, \sigma_{it2} = (SK_{t2}^u)^{\frac{1}{\Delta_{t,S_u(0)}}} \cdot g^{r'_t}$$

If $t \in S_u$, the participant computes $\hat{q}'_u(t)$ and

$$\sigma_{it1} = H_1(t)^{r'_t} \cdot \left(\prod_{j=1}^s u_j^{m_{ij}} \cdot \mathcal{H}_i \right)^{\hat{q}_u(t) + \hat{q}'_u(t)}, \sigma_{it2} = g^{r'_t}.$$

The participant sets $\tau_0 = \text{name} || u_1 || \dots || u_s$ and computes the signature of τ_0 as $ABS.\text{Sign}(\tau_0)$ with $skey$ and sets the tag of m as $\tau = \tau_0 || ABS.\text{Sign}(\tau_0)$. After the above computation, the participant u uploads the data m , authenticators $\sigma = \{\sigma_{it1}, \sigma_{it2}\}_{1 \leq i \leq n, t \in S}$, and tag τ to the cloud server for sharing.

- 2) *Modification*(ω_u, Com, mpk, SK_u): A legitimate participant in the group can modify the shared data stored on the cloud server, including updating, inserting, and deleting. Specifically, the participant generates the modified data block, and computes the index and new authenticator for the modified data block. After that, the participant uploads the modified data block, index, and new authenticator to the cloud server along with the modification type. Since the data is divided into n blocks, the data modification is performed at the block level. The legitimate participant can modify one or more data blocks at a time. When modifying multiple data blocks, the participant can upload all data blocks and corresponding modification types to the cloud server at one time after generating new authenticators and indexes of all modified data blocks. The specific operations are as follows.

Updating: When updating the existing data block m_i to m'_i , the participant first generates the new random number $\mathcal{R}'_i$ because $\mathcal{R}'_i$ is computed based on the content of the data block. After that, the participant computes new authenticators $\{\sigma'_{it1}, \sigma'_{it2}\}_{t \in S}$ for m'_i . The participant sets the modification command Com as $\{m'_i, (\mathcal{V}_i, \mathcal{R}'_i), \{\sigma'_{it1}, \sigma'_{it2}\}_{t \in S}, U\}$ where U indicates the updating operation.

Inserting: When inserting a new data block m'_i at the index i , the participant first generates the virtual number $\mathcal{V}'_i$ to remain the right order of the data blocks. Similar to the updating operation, the participant then generates the new random number $\mathcal{R}'_i$. After that, the participant generates the authenticators $\{\sigma'_{it1}, \sigma'_{it2}\}_{t \in S}$ for m'_i to be inserted.

The participant sets the modification command Com as $\{m'_i, (\mathcal{V}'_i, \mathcal{R}'_i), \{\sigma'_{it1}, \sigma'_{it2}\}_{t \in S}, I\}$ where I indicates the inserting operation.

Deleting: When deleting the data block m_i , the participant sets the deletion version $m'_i = m_i || \text{deletion}$, which is a new data block. Similar to the updating operation, the participant generates the new random number $\mathcal{R}'_i$ and new authenticators $\{\sigma'_{it1}, \sigma'_{it2}\}_{t \in S}$ for m'_i . The participant sets the modification command Com as $\{m'_i, (\mathcal{V}_i, \mathcal{R}'_i), \{\sigma'_{it1}, \sigma'_{it2}\}_{t \in S}, D\}$ where D indicates the deleting operation. It is worth noting that when the deleting operation is executed, the participant generates a new version m'_i for the data block m_i to be deleted and computes the authenticators of m'_i . After receiving the deleting request, the cloud server first verifies the validity of the authenticators $\{\sigma'_{it1}, \sigma'_{it2}\}_{t \in S}$. If they are valid, which indicates the deleting request is proposed by a legitimate participant, the cloud server performs the deleting operation. In this way, the illegal participants can be prevented from deleting the shared data.

After generating the modification command Com , the participant sends it to the cloud server and requests the cloud server to modify the shared data. The cloud server finds the corresponding location in the index-hash table according to the index i and then performs corresponding modification operations according to the modification type in Com , including updating, inserting, or deleting the data block m_i , authenticators $\{\sigma_{it1}, \sigma_{it2}\}_{t \in S}$, and $(\mathcal{V}_i, \mathcal{R}_i)$ stored in the index-hash table.

The *Verification* module includes the algorithms of *ChalGen*, *ProofGen*, and *CheckCert*.

- 1) *ChalGen*(mpk): After receiving the integrity checking and reliability authentication request issued by a participant, the TPA first chooses a subset C with c elements to specify the indexes of the challenged data blocks from the set $[1, n]$, and randomly chooses $\{v_i \in \mathbb{Z}_p^*\}_{i \in C}$. Finally, the TPA delivers $\{i, v_i\}_{i \in C}$ as *Chal* to the cloud server.
- 2) *ProofGen*(*Chal*, σ, m): With the stored data m and corresponding authenticators σ , the cloud server computes

$$\sigma_{t1} = \prod_{i \in C} \sigma_{it1}^{v_i}, \sigma_{t2} = \prod_{i \in C} \sigma_{it2}^{v_i}, \mu_j = \sum_{i \in C} v_i \cdot m_{ij}.$$

The cloud server sends the identifiers of the challenged data blocks $\{\mathcal{V}_i, \mathcal{R}_i\}_{i \in C}$, tag τ , and $\{\sigma_{t1}, \sigma_{t2}, \mu_j\}_{1 \leq j \leq s, t \in S}$ as storage proof *Prf* to TPA.

- 3) *CheckCert*($mpk, Chal, Prf$): After receiving *Prf*, the TPA first verifies the validity of τ . If τ is valid, the TPA performs the integrity checking and reliability authentication for the shared data. The TPA verifies whether the following equation is correct:

$$\prod_{t \in S} \left(\frac{e(g, \sigma_{t1})}{e(g, \sigma_{t2}, H_1(t))} \right)^{\Delta_{t,S(0)}} \stackrel{?}{=} e(g_1, g_2)^{\sum_{i \in C} v_i}.$$

$$e \left(Y, \prod_{j=1}^s u_j^{\mu_j} \cdot \prod_{i \in C} H_3(\text{name} || \mathcal{V}_i || \mathcal{R}_i)^{v_i} \right). \quad (1)$$

If yes, the TPA returns “True” to the participant issuing the request, which indicates that the shared data is intact and shared by legitimate participants in the group. Otherwise, the TPA outputs “False”.

D. Identity Traceability of SEAYS

In SEAYS, before storing the received data, the cloud server requests the TPA to verify the integrity and the reliability of the data. If the TPA returns “True”, the cloud server stores the received data. This step has the following two functions. First, this process prevents the cloud server from storing corrupted data by verifying the integrity of the received data, thus avoids possible disputes between the cloud server and participants. Besides, this process can verify whether the data received by the cloud server is uploaded by legitimate participants, which indirectly verifies the identity of data uploader.

In SEAYS, we assume that legitimate participants honestly modify the shared data. But in some actual scenarios, the participants in the sharing group may have bad behaviors, such as maliciously modifying the data. At this time, the above method cannot track the identity of malicious participants. In SEAYS, the identities of participants are fully anonymous to the cloud server, TPA and other participants in the group, so it is infeasible to directly track the identities of malicious participants. Therefore, based on the above basic construction, we make the following additional improvements to support the application of SEAYS in the scenario of identity tracking.

When modifying the data block m_i , the participant computes the hash $hash(m_i)$ of modified block by using the collision-resistance hash function H_2 , and obtains the system time $time_i$. After that, the participant uploads $hash(m_i)$, $time_i$, as well as the modification command Com generated in *Modification* algorithm to the cloud server. In addition, the hash values of all modified data blocks are reserved locally by the participants for use in the identity tracking process.

Since the cloud server performs all data modifications, a list recording the modification history of each data block can be maintained at the same time. For the shared data with the identifier $name$, the cloud server keeps a list HIS , which stores tuples $(\mathcal{V}_i, \mathcal{R}_i, time_i, type, hash_i)$, where $(\mathcal{V}_i, \mathcal{R}_i)$ uniquely represents the modified data block, $type$ indicates the modification operation, such as *Updating*, *Inserting*, and *Deleting*, $time_i$ and $hash_i$ represent respectively the system time and hash value received from the participant.

In order to record the correspondence between the modification operation and the participant identity, a list MOD for data with the identifier $name$ should be maintained by the attribute authority. Specifically, the list includes tuples $(\omega_u, \mathcal{V}_i, \mathcal{R}_i, time_i)$, where ω_u is the attribute set of the participant, $(\mathcal{V}_i, \mathcal{R}_i)$ uniquely represents the modified data block, and $time_i$ is the system time when the modification is performed. After each modification, the participant generates the corresponding tuple and sends it to the attribute authority for recording. Since the attribute authority issues the secret key to the participant in *KeyGen* algorithm, the tuple will not disclose the additional identity information of the participant. In addition, the tuple only contains the index of the data block, and will

not disclose the information of the shared data to the attribute authority.

When data block m_i is found to be maliciously modified, the participant accessing the shared data feeds back $(\mathcal{V}_i, \mathcal{R}_i)$ and hash value $hash(m_i)$ of m_i to the attribute authority. Next, the attribute authority sends the received $(\mathcal{V}_i, \mathcal{R}_i)$ and $hash(m_i)$ to the cloud server and requests to return the modification history of the corresponding data block. After receiving the request, the cloud server searches the maintained list HIS to find the responding record $(\mathcal{V}_i, \mathcal{R}_i, time_i, type, hash_i)$ and returns to the attribute authority. Thus the attribute authority finds the corresponding modification type $type$ and modification time $time_i$. Further, the attribute authority looks up the tuples corresponding to $(\mathcal{V}_i, \mathcal{R}_i, time_i)$ from the list MOD , finds the corresponding participant attribute set ω_u , and then tracks the identity of the participant who maliciously modified the data block m_i . Since the malicious behavior is fed back by participants in the group, when there is a dispute, the tracked participant can use the hash values of all modified data blocks stored by himself as evidence to raise an objection to the attribute authority and request arbitration.

E. Identity Revocation of SEAYS

When performing malicious behaviours or requesting to exit data sharing, the participant should be revoked to stop data modification and data access. Therefore, based on the above construction, we make the following additional improvements to support the application of SEAYS in the scenario of identity revocation. Specifically, the attribute authority issues a periodic key to each legitimate participant. When performing data modification and data access, the participant uses this periodic key to generate a signature for the cloud server to verify identity validity. When a malicious participant appears or the time is up to the next period, the attribute authority updates the periodic key and distributes it to all legitimate participants to achieve identity revocation.

At the *Initialization* module of SEAYS, the attribute authority generates secret key $\theta_A \in Z_p^*$ and the corresponding public key $pk_A = g^{\theta_A}$. Then, pk_A is released as the public parameter, and θ_A is kept secret for periodic key generation and updating. Besides, the attribute authority selects a fixed time interval as the time period $timp$ for updating the periodic key.

At the beginning of time period $timp$, the attribute authority computes $timp = timp + 1$ and updates the periodic key as $ssk_{timp} = H_1(timp)^{\theta_A}$, then distributes ssk_{timp} to all legitimate participants in the sharing group. It is worth noting that when a malicious participant or an outgoing participant appears, the attribute authority sets $timp = timp + 1$ and starts the above periodic key updating process. By flexibly combining periodic updating and timely updating, on the one hand, malicious participants can be revoked in time, and on the other hand, the risk of periodic key being attacked and leaked can be avoided.

In order to perform data modification or data access, the participant randomly chooses $r \in Z_p^*$ and computes $R = g^r$. During data modification, the participant uses the current periodic key to generate a signature $\delta_{timp} = H_3(timp || name || \mathcal{V}_i || \mathcal{R}_i)^r$.

ssk_{timp} , then uploads R , δ_{timp} and the modification command Com to the cloud server. When requesting data access, the participant generates the signature as $\delta_{timp} = H_3(timp||name||access)^r \cdot ssk_{timp}$, then uploads R and δ_{timp} to the cloud server.

After receiving the data modification or access request submitted by the participant, the cloud server first verifies the participant identity. For data modification request, the cloud server checks whether the following (2)

$$e(g, \delta_{timp}) = e(pk_A, H_1(timp)) \cdot e(R, H_3(timp||name||\mathcal{V}_i||\mathcal{R}_i)) \quad (2)$$

is true. If yes, which indicates that the participant is valid and has not been revoked, the cloud server accepts the modification request. Otherwise, the cloud server rejects the modification request. Similarly, for data access request, the cloud server verifies whether the following (3)

$$e(g, \delta_{timp}) = e(pk_A, H_1(timp)) \cdot e(R, H_3(timp||name||access)) \quad (3)$$

is true. If yes, which indicates that the participant is legitimate and has not been revoked, the cloud server allows the participant to download the shared data. Otherwise, access to the shared data is prohibited. The above process, on the one hand, realizes the identity revocation of malicious participants and participants who are about to exit the data sharing, on the other hand, it also realizes access control of the shared data.

In addition, when requesting to perform the integrity checking and reliability authentication for the shared data, the participant needs to deliver a request to TPA. In order to verify the validity of the participant identity and remain the full identity anonymity, the participant is required to use the periodic key to compute the signature $\delta_{req} = H_3(timp||name||auditingreq)^r \cdot ssk_{timp}$ for message *auditingrequest* and send to the TPA. The TPA then uses public key pk_A to verify whether the (4)

$$e(g, \delta_{req}) = e(pk_A, H_1(timp)) \cdot e(R, H_3(timp||name||auditingreq)) \quad (4)$$

is true. If yes, which indicates that the participant is legitimate and has not been revoked, the TPA generates the challenge for the request and sends to the cloud server as described in the *Verification* module. Since the verification does not require participant identity, the full identity anonymity is remained.

V. PERFORMANCE ANALYSIS

In this section, we evaluate the performance of SEAYS through functionality comparison, numerical analysis, and efficiency comparison.

A. Functionality Comparison

Since the work in [5], [6], [7], [8] is recently proposed to investigate anonymous integrity checking for group shared data and is similar to SEAYS, we compare the functionalities of SEAYS with the schemes reported in [5], [6], [7], [8]. As shown

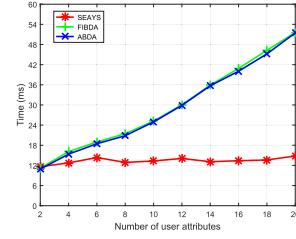


Fig. 7. Time of setup.

in Table I, SEAYS supports data modification performed by multiple participants in cloud data sharing, and keeps the participant identities fully anonymous. In addition, SEAYS is constructed based on ABS, which reduces the burden of certificate management. Since the participants have to use the secret keys that are related to their identities to compute the authenticators for the data uploaded during data modification, and the participants also need the corresponding identity information to join the sharing group, the identities of the participants in [5], [6], [7], [8] are known by at least one party, such as the group manager.

B. Efficiency Comparison

In this section, we implement three modules *Initialization*, *Sharing*, and *Verification* of SEAYS. Since the work in [19] and [20] is about the attribute-based integrity checking scheme for cloud storage, and the work in [7] and [8] is recently proposed to investigate anonymous integrity checking for group shared data, all of them are related to the work in this paper. Therefore, we compare the performance of SEAYS with those reported in [7], [19], [20] and [8]. For brevity, we denote the schemes in [7], [19], [20] and [8] as FIBDA, ABDA, CoRPA, and PRAYS respectively. FIBDA associates the user's identity with a set of biometric attributes, and we view it as an attribute-based integrity checking scheme.

The implementation is conducted on Ubuntu 14.04 (64-bit) with Intel(R) Core(TM) i5-4590 @ 3.30 GHz CPU with 2 GB memory. We use the PBC Library (0.5.14) [21] to implement the cryptographic computation. To be consistent with the implementation of the schemes being compared, we choose the efficient asymmetric bilinear pairing $\mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ under d159.param when comparing the performance of SEAYS with ABDA and FIBDA and the symmetric bilinear pairing $\mathbb{G}_1 \times \mathbb{G}_1 \rightarrow \mathbb{G}_T$ under a.param when comparing the performance of SEAYS with CoRPA and PRAYS. In implementation, we set $|S| = 10$, $d = 2$ in the predicate $\mathcal{T}_{d,S}$, and the number of sectors in each data block $s = 100$. We skip the challenge generation algorithm in the comparison because the algorithm's computation in all schemes is the same.

First, we compare the performance of SEAYS with FIBDA and ABDA in Figs. 7–11. Because FIBDA and ABDA are attribute-based integrity checking schemes dealing with the static data from a single owner, we call the users of cloud storage service as “users”. TU and TA represent the number of attributes of user and TPA respectively, and both range from 2 to 20 in the comparison. We compare the time consumption of the three

TABLE I
FUNCTIONALITY COMPARISON

Schemes	Reducing Certificate Management	Supporting Data Modification	Anonymity To Cloud	Anonymity To TPA	Anonymity To participants
Rabaninejad et al. [7]	No	Yes	No	Yes	No
Wang et al. [5]	No	Yes	No	No	No
He et al. [8]	No	Yes	Yes	Yes	No
Hu et al. [6]	No	No	Yes	Yes	No
SEAYS	Yes	Yes	Yes	Yes	Yes

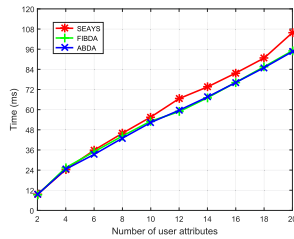


Fig. 8. Time of key generation.

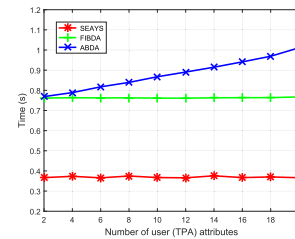


Fig. 11. Time of integrity checking.

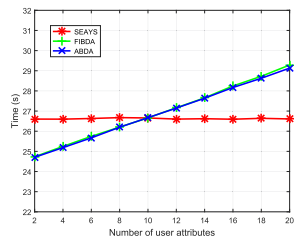
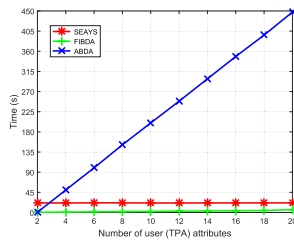
Fig. 9. Time of *AuthGen*.

Fig. 10. Time of proof generation.

schemes as TU increases. In this comparison, the number of data blocks is $n = 512$.

Fig. 7 represents the time consumption of the setup algorithm. Setup algorithms in FIBDA and ABDA compute parameters for each attribute, so the time consumption grows as TU increases. The setup algorithm in SEAYS only generates the master private key and master public key of the system. Therefore, the time consumption of SEAYS remains unchanged and is lower than those of FIBDA and ABDA.

It can be seen that from Fig. 8 the time consumption of the key generation algorithm in the three schemes grows linearly as TU increases. The reason is that algorithms for key generation in the

three schemes compute secret keys for all user attributes. The algorithms in the three schemes contain three exponentiations, which consume the most time, causing the time consumption to grow at similar rates.

Fig. 9 represents the relationship between the time consumption of the authenticator generation algorithm and TU. In SEAYS, the algorithm computes the authenticators for all attributes in the predicate so the time consumption remains unchanged. In FIBDA and ABDA, the algorithm generates the authenticators for user attributes, causing the time consumption to grow as TU rises. As one shall see, the performance of SEAYS outperforms FIBDA and ABDA as TU increases. It is meaningful to reduce the time consumption of this algorithm since the authenticator generation is the most resource-intensive and the user is usually resource-limited.

Fig. 10 demonstrates the time consumption of the proof generation algorithm. In addition to the storage proof, the algorithm in ABDA needs to compute the authenticators for the attributes of TPA. Therefore, the time consumption of ABDA grows as TA increases and is much higher than those of SEAYS and FIBDA. The algorithm in SEAYS generates the storage proof for all attributes in the predicate, so the time consumption is independent of TU. The algorithm in FIBDA generates the storage proof for the user attributes meaning the computation time grows as TU increases. Because the algorithm in SEAYS performs more exponentiations than FIBDA, the time consumption of SEAYS is slightly higher than FIBDA.

Obviously, the time consumption of the integrity checking algorithm in SEAYS and FIBDA is independent of TU in Fig. 11. This phenomenon can be explained as follows: the algorithm in SEAYS performs the integrity checking with all attributes in the predicate, and the algorithm in FIBDA performs the integrity checking with the d -element subset of the user attributes. The

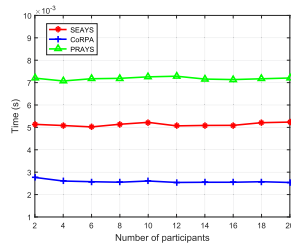


Fig. 12. Time of setup.

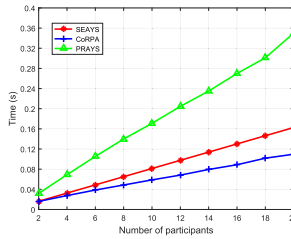


Fig. 13. Time of key generation.

algorithm in ABDA performs the integrity checking with the TPA's attributes, so the time consumption grows with TA. Furthermore, SEAYS outperforms FIBDA and ABDA in terms of integrity checking since the algorithms in FIBDA and ABDA perform more pairings and exponentiations than SEAYS.

Because the schemes in SEAYS, CoRPA and PRAYS deal with the situation where a group of participants share the data stored on the cloud server, we evaluate the performance when the number of participants increases in Figs. 12 and 13. In this comparison, we set the number of participant attributes to be 2. Since the time consumed by the algorithms of authenticator generation, proof generation, and integrity checking is irrelevant to the number of participants, we only need to compare the performance of the setup algorithm and key generation algorithm.

We can see from Fig. 12 that the time consumption of SEAYS is a little higher than that of CoRPA because there exists two exponentiations in the setup algorithm. PRAYS takes the most time since it executes two exponentiations and one pairing.

Fig. 13 shows the time consumption of key generation algorithm. Obviously, the time consumption of the three schemes grows linearly as the number of participants increases. During the key generation, three exponentiations are executed in SEAYS to distribute the secret key for each participant attribute, thus SEAYS costs a little more time than CoRPA. Obviously, PRAYS costs the most time because it needs to execute two exponentiations and four pairings for each participant's secret key.

Now, we set the number of participants in the group to be 2 and compare the performance of the algorithms of authenticator generation, proof generation, and integrity checking with the number of data blocks ranging from 100 to 1000 in Figs. 14–16 because the time consumed by the three algorithms is related to the number of data blocks.

Fig. 14 represents the relationship between the time consumption of the authenticator generation algorithm and number of data blocks. In SEAYS, the participant needs to compute $|S|$

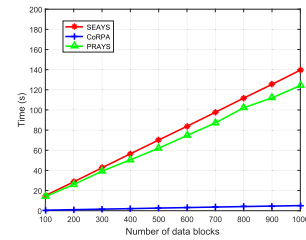


Fig. 14. Time of AuthGen.

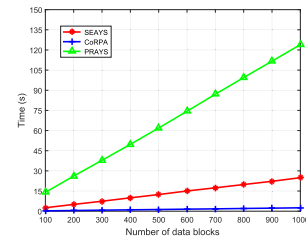


Fig. 15. Time of proof generation.

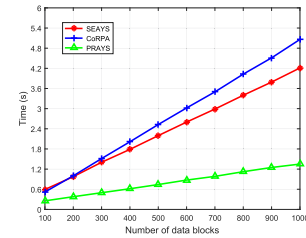


Fig. 16. Time of integrity checking.

authenticators for each data block. In this implementation, the participant needs to compute 10 authenticators for each data block, so the participant performs more exponentiation and multiplication operations on $\mathbb{G}_1$. Thus, we can observe in Fig. 14 that SEAYS costs a little more time than PRAYS. Fortunately, the efficiency of the authenticator generation algorithm is acceptable since the participant only computes the authenticators once for each data block. In addition, PRAYS can achieve more comprehensive identity privacy protection.

For simplicity, we assume that the TPA requires the cloud server to generate the storage proof for all data blocks in the challenge phase. Fig. 15 shows the time consumption of the proof generation algorithm. It is clear that SEAYS consumes much less time than PRAYS. The reasons are as follows. The cloud server in PRAYS has to execute one exponentiation for each sector. In this implementation, the cloud server has to execute 100 exponentiations for each data block. In SEAYS, the most time-consuming operation for each data block performed by the cloud server is two exponentiation. As shown in Fig. 15, when the number of data blocks is 1000, PRAYS consumes 123.96 s, while SEAYS consumes 24.94 s and CoRPA consumes 2.5 s. SEAYS only takes a little more time than CoRPA. The high efficiency of this algorithm is meaningful because the cloud server needs to compute the storage proof frequently.

Fig. 16 demonstrates the time consumption of the integrity checking algorithm. We can see that CoRPA consumes more

time than SEAYS and PRAYS because the TPA needs to perform two exponentiations for each challenged data block in CoRPA. Although SEAYS consumes more time than PRAYS as shown in Fig. 16, the TPA can afford it because it is a third party dedicated to checking the integrity of the shared data.

From the above performance comparison, we can see that the efficiency of SEAYS is acceptable on the whole. Although the overall performance does not outperform that of the other schemes, the efficiency is still good in some phases, such as setup in Fig. 7 and integrity checking in Fig. 11. Moreover, except the key generation algorithm, the time consumption of SEAYS does not increase as the number of participant attributes grows, which is advantageous when the number of participant attributes is large. In addition, SEAYS can achieve full anonymity of participant identity, while the other schemes cannot. Since the identity privacy is a concern of participants, it is worth achieving full identity anonymity at the expense of the performance of some algorithms.

VI. RELATED WORK

In this section, we briefly review existing integrity checking schemes related to this study.

The work in [22], [23], [24] initially studied data integrity checking of cloud storage. Ateniese et al. first proposed Provable Data Possession (PDP), which allows the user to check the integrity of data stored on the cloud server without downloading the data. According to the principle of random sampling, the user only needs to randomly select a certain number of data blocks to check the integrity of the entire file, thus minimizing communication overhead [22]. By using error-correcting codes, Juels et al. proposed Proofs of Retrievability (PoR) to realize data integrity checking and data retrievability at the same time [23]. Subsequently, a vast number of relevant studies were conducted in different directions, e.g., the integrity checking of the dynamic data and the integrity checking of the shared data.

In cloud storage, the modification of the stored data is an operation that cloud users frequently perform, including updating the stored data, inserting new data and deleting useless data with the help of the cloud server. Researchers then focus on the integrity checking of this dynamic data [5], [25], [26], [27], [28], [29], [30], [31]. To record the information about the modification and keep a mapping between the data indices and authenticator indices, Tian et al. designed a two-dimensional data structure called a dynamic hash table located at the TPA in [25]. In order to avoid the re-computation of authenticators caused by the change of block indices due to the insertion and deletion of data, Jin et al. differentiated between authenticator index (used for authenticator computation) and block index (indicate block position), and introduced an index switcher to maintain the mapping between them in [26]. Similarly, to solve the problem of block index changes, Shang et al. used a Merkle hash tree to authenticate block authenticators in [27]. Considering the dynamic operation of multicopy data, Zhou et al. designed a novel dynamic structure that improves the Merkle Hash Tree and constructed an integrity checking scheme,

which ensures the consistency of replica updates in [28], [31]. Furthermore, Shen et al. designed a new dynamic structure, including a doubly linked info table and a location array, to solve the relationship between data blocks and their locations, while reducing the computational and communication overheads [29]. In order to support efficient dynamic operations and realize data retrievability, Sengupta et al. proposed a dynamic PoR scheme using homomorphic hash technology [30]. Different from the work described above, the method in [5] utilized the index hash table to support the modification of the shared data. In [5], all participants in the group can modify the shared data.

Data sharing is an important service provided by cloud storage, and a vast number of subsequent studies devoted to the integrity checking of the shared data [32], [33], [34], [35], [36], [37], [38]. Since data privacy is a concern of participants, Gao et al. [38] considered the integrity of the encrypted data. Although encryption is a common method to protect data privacy, such as searchable encryption [39], convergent-encryption [40], and order-revealing encryption [41], it will increase the computational burden when the participants access the shared data. Therefore, instead of encrypting the whole data, Shen et al. proposed a data integrity checking scheme that hides the sensitive information included in the shared data in [34]. In their scheme, the data blocks containing sensitive information were first blinded and then sent to a sanitizer. The sanitizer sanitizes the blinded data blocks into a uniform format. Nevertheless, the schemes in [34] and [38] did not consider the enrollment of new participants. Generally the existing work relies on a group manager to enroll new participants by issuing the secret keys, which ensures the reliability of the shared data. In [35], Yuan et al. employed the technique of polynomial-based authentication tags to propose an integrity checking scheme that allows multiple participants to modify the shared data, where the owner of the shared data acted as the group manager and was responsible for enrolling new participants. Additionally, Tian et al. considered the issue of multiple data owners and introduced a group manager delegated by multiple data owners to manage the group [36]. To avoid the problems of certificate management and key escrow, Li et al. proposed an integrity checking scheme of group shared data based on the certificateless signature technique and employed the original creator of the group as the group manager who enrolls new participants by generating partial keys for them [33]. Unlike the above work, Fu et al. constructed a homomorphic verifiable group signature and proposed a public integrity checking scheme with multiple group managers in the sharing group. In the scheme, any group manager can modify the shared data and enroll new participants, which avoids the abuse of single-authority power [37]. In view of the problem that group managers have lots of permissions in the existing literature, Huang et al. distributed the power of the manager to group users and proposed a threshold signature based integrity checking scheme for non-manager groups [32]. In their scheme, each participant has the same ability to generate the secret key and signatures, and there is no group manager in the sharing group, which avoids the problem of power concentration. Unfortunately, their scheme does not support the enrollment of

new participants, nor does it consider the identity privacy of participants.

Some existing studies considered the identity protection of participants. Rabaninejad et al. hid the signer's identity by employing the proxy re-signature technique to construct the integrity checking scheme for shared data in [7]. Specifically, the signer first uses his own secret key to generate the first-level signature of the data block and sends it to the cloud server. Then, the cloud server uses the re-signature key to convert the received signature into a second-level signature that is not associated with the signer's identity. Thus, the signer's identity is anonymous to the TPA. However, the signer's identity is known by the cloud server and group manager. To enhance the identity protection of participants, He et al. designed the blockless Merkle tree as the authenticated structure of data integrity checking, whose construction process only requires the data blocks and does not require any secret keys. In addition, He et al. designed the permission-based signature technique to compute the signature for the metadata of the authenticated structure, and the verification of the signature does not require the public key related to the participant's identity. Therefore, their scheme achieves identity anonymity in an out-of-group scope, that is, the cloud server and TPA could not learn the group participant's identity [8]. However, the participant identities were known by the group manager and other participants in the group in [8]. Similarly, Hu et al. used the improved group signature technique to construct an integrity checking scheme, where the participant anonymously shares data with the cloud server. However, the group manager needs to manage group memberships and issue secret keys for new participants by constructing the polynomial function, and thus knows the participant's real identity in [6]. Besides, since no certificate management is needed in ABS, Li et al. applied an ABS scheme to the data integrity checking to simplify the key management [19]. However, the participant's identity was not protected in their scheme. In [20], the study considered the identity privacy of a signer. The cloud server was responsible for preventing the TPA from knowing the signer's identity. However, [20] only dealt with the static data from a single owner and the cloud server knew the signer's real identity. These cases prove that the identity anonymity of the participant is an urgent problem to be solved.

VII. CONCLUSION

This paper proposed a novel approach, named SEAYS, to achieve fully anonymous integrity checking and reliability authentication for cloud data sharing. The participant's identity remains anonymous to the cloud server, TPA, and all group participants since the cloud data sharing is implemented under all attributes in the designed predicate. Furthermore, the reliability of data sources is effectively ensured without compromising participant identity anonymity. Besides, the identity traceability and identity revocation is considered in SEAYS. A security proof under the random oracle model demonstrated the unforgeability and indistinguishability of SEAYS. The results of performance analysis showed that the efficiency of SEAYS is not very excellent. Therefore, the future work is to study a

lightweight integrity checking method for dynamic shared data with full identity anonymity.

ACKNOWLEDGMENTS

The authors would like to thank the editor-in-chief, associate editor and the anonymous reviewers sincerely for their valuable feedback.

REFERENCES

- [1] R. Ghasemi, M. M. Al Aziz, N. Mohammed, M. H. Dehkordi, and X. Jiang, "Private and efficient query processing on outsourced genomic databases," *IEEE J. Biomed. Health Informat.*, vol. 21, no. 5, pp. 1466–1472, Sep. 2017.
- [2] S. Wang et al., "HEALER: Homomorphic computation of exact logistic regression for secure rare disease variants analysis in GWAS," *Bioinformatics*, vol. 32, no. 2, pp. 211–218, 2016.
- [3] H. Cho, D. J. Wu, and B. Berger, "Secure genome-wide association analysis using multiparty computation," *Nature Biotechnol.*, vol. 36, no. 6, pp. 547–551, 2018.
- [4] W. Xie et al., "SecureMA: Protecting participant privacy in genetic association meta-analysis," *Bioinformatics*, vol. 30, no. 23, pp. 3334–3341, 2014.
- [5] B. Wang, B. Li, and H. Li, "Oruta: Privacy-preserving public auditing for shared data in the cloud," *IEEE Trans. Cloud Comput.*, vol. 2, no. 1, pp. 43–56, Jan.–Mar. 2014.
- [6] A. Hu, R. Jiang, and B. Bhargava, "Identity-preserving public integrity checking with dynamic groups for cloud storage," *IEEE Trans. Serv. Comput.*, vol. 14, no. 4, pp. 1097–1110, Jul./Aug. 2021.
- [7] R. Rabaninejad, M. Ahmadian, M. Rajabzadeh Asaar, and M. R. Aref, "A lightweight auditing service for shared data with secure user revocation in cloud storage," *IEEE Trans. Serv. Comput.*, vol. 15, no. 1, pp. 1–15, Jan.–Feb. 2022.
- [8] K. He, J. Chen, Q. Yuan, S. Ji, D. He, and R. Du, "Dynamic group-oriented provable data possession in the cloud," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 3, pp. 1394–1408, May/Jun. 2021.
- [9] M. Z. Hasan, M. S. R. Mahdi, M. N. Sadat, and N. Mohammed, "Secure count query on encrypted genomic data," *J. Biomed. Informat.*, vol. 81, pp. 41–52, 2018. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1532046418300455>
- [10] G. Kale, E. Ayday, and O. Tastan, "A utility maximizing and privacy preserving approach for protecting kinship in genomic databases," *Bioinformatics*, vol. 34, no. 2, pp. 181–189, 2018.
- [11] J. L. Raisaro et al., "MedCo: Enabling secure and privacy-preserving exploration of distributed clinical and genomic data," *IEEE/ACM Trans. Comput. Biol. Bioinf.*, vol. 16, no. 4, pp. 1328–1341, Jul. 2019.
- [12] J. Li and K. Kim, "Attribute-based ring signatures," *IACR Cryptol. ePrint Arch.*, vol. 2008, 2008, Art. no. 394.
- [13] J. Sun, Y. Su, J. Qin, J. Hu, and J. Ma, "Outsourced decentralized multi-authority attribute based signature and its application in IoT," *IEEE Trans. Cloud Comput.*, vol. 9, no. 3, pp. 1195–1209, Jul.–Sep. 2019.
- [14] J. Li, M. H. Au, W. Susilo, D. Xie, and K. Ren, "Attribute-based signature and its applications," in *Proc. 5th ACM Symp. Inf. Comput. Commun. Secur.*, New York, NY, USA: Association for Computing Machinery, 2010, pp. 60–69, doi: [10.1145/1755688.1755697](https://doi.org/10.1145/1755688.1755697).
- [15] D. Boneh and M. Franklin, "Identity-based encryption from the weil pairing," in *Advances in Cryptology*, J. Kilian, Ed., Berlin, Germany: Springer, 2001, pp. 213–229.
- [16] V. Shoup, "Lower bounds for discrete logarithms and related problems," in *Advances in Cryptology*, W. Fumy, Ed., Berlin, Germany: Springer, 1997, pp. 256–266.
- [17] E.-J. Goh and S. Jarecki, "A signature scheme as secure as the diffie-hellman problem," in *Advances in Cryptology*, E. Biham, Ed., Berlin, Germany: Springer, 2003, pp. 401–415.
- [18] D. Cash, E. Kiltz, and V. Shoup, "The twin diffie-hellman problem and applications," in *Advances in Cryptology*, N. Smart, Ed., Berlin, Germany: Springer, 2008, pp. 127–145.
- [19] Y. Li, Y. Yu, G. Min, W. Susilo, J. Ni, and K. R. Choo, "Fuzzy identity-based data integrity auditing for reliable cloud storage systems," *IEEE Trans. Dependable Secure Comput.*, vol. 16, no. 1, pp. 72–83, Jan./Feb. 2019.

- [20] Y. Yu, Y. Li, B. Yang, W. Susilo, G. Yang, and J. Bai, "Attribute-based cloud data integrity auditing for secure outsourced storage," *IEEE Trans. Emerg. Topics Comput.*, vol. 8, no. 2, pp. 377–390, Apr.–Jun. 2020.
- [21] B. Lynn, "Pairing-based cryptography," Jun. 2013. [Online]. Available: <https://crypto.stanford.edu/pbc>
- [22] G. Ateniese et al., "Provable data possession at untrusted stores," in *Proc. 14th ACM Conf. Comput. Commun. Secur.*, New York, NY, USA: Association for Computing Machinery, 2007, pp. 598–609, doi: [10.1145/1315245.1315318](https://doi.org/10.1145/1315245.1315318).
- [23] A. Juels and B. S. Kaliski, "PORs: Proofs of retrievability for large files," in *Proc. 14th ACM Conf. Comput. Commun. Secur.*, New York, NY, USA: Association for Computing Machinery, 2007, pp. 584–597, [Online]. Available: <https://doi.org/10.1145/1315245.1315317>
- [24] H. Shacham and B. Waters, "Compact proofs of retrievability," in *Advances in Cryptology*, J. Pieprzyk, Ed., Berlin, Germany: Springer, 2008, pp. 90–107.
- [25] H. Tian et al., "Dynamic-hash-table based public auditing for secure cloud storage," *IEEE Trans. Serv. Comput.*, vol. 10, no. 5, pp. 701–714, Sep./Oct. 2017.
- [26] H. Jin, H. Jiang, and K. Zhou, "Dynamic and public auditing with fair arbitration for cloud data," *IEEE Trans. Cloud Comput.*, vol. 6, no. 3, pp. 680–693, Jul.–Sep. 2018.
- [27] T. Shang, F. Zhang, X. Chen, J. Liu, and X. Lu, "Identity-based dynamic data auditing for Big Data storage," *IEEE Trans. Big Data*, vol. 7, no. 6, pp. 913–921, Dec. 2021.
- [28] L. Zhou, A. Fu, Y. Mu, H. Wang, S. Yu, and Y. Sun, "Multicopy provable data possession scheme supporting data dynamics for cloud-based electronic medical record system," *Inf. Sci.*, vol. 545, pp. 254–276, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0020025520307982>
- [29] J. Shen, J. Shen, X. Chen, X. Huang, and W. Susilo, "An efficient public auditing protocol with novel dynamic structure for cloud data," *IEEE Trans. Inf. Forensics Secur.*, vol. 12, no. 10, pp. 2402–2415, Oct. 2017.
- [30] B. Sengupta and S. Ruj, "Efficient proofs of retrievability with public verifiability for dynamic cloud storage," *IEEE Trans. Cloud Comput.*, vol. 8, no. 1, pp. 138–151, Jan.–Mar. 2020.
- [31] L. Zhou, A. Fu, J. Feng, and C. Zhou, "An efficient and secure data integrity auditing scheme with traceability for cloud-based EMR," in *Proc. IEEE Int. Conf. Commun.*, 2020, pp. 1–6.
- [32] L. Huang et al., "IPANM: Incentive public auditing scheme for non-manager groups in clouds," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 2, pp. 936–952, Mar./Apr. 2022.
- [33] J. Li, H. Yan, and Y. Zhang, "Certificateless public integrity checking of group shared data on cloud storage," *IEEE Trans. Serv. Comput.*, vol. 14, no. 1, pp. 71–81, Jan./Feb. 2021.
- [34] W. Shen, J. Qin, J. Yu, R. Hao, and J. Hu, "Enabling identity-based integrity auditing and data sharing with sensitive information hiding for secure cloud storage," *IEEE Trans. Inf. Forensics Secur.*, vol. 14, no. 2, pp. 331–346, Feb. 2019.
- [35] J. Yuan and S. Yu, "Efficient public integrity checking for cloud data sharing with multi-user modification," in *Proc. IEEE Conf. Comput. Commun.*, 2014, pp. 2121–2129.
- [36] H. Tian, F. Nan, H. Jiang, C.-C. Chang, J. Ning, and Y. Huang, "Public auditing for shared cloud data with efficient and secure group management," *Inf. Sci.*, vol. 472, pp. 107–125, 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0020025518306996>
- [37] A. Fu, S. Yu, Y. Zhang, H. Wang, and C. Huang, "NPP: A new privacy-aware public auditing scheme for cloud data sharing with group users," *IEEE Trans. Big Data*, vol. 8, no. 1, pp. 14–24, Feb. 2022.
- [38] X. Gao, J. Yu, Y. Chang, H. Wang, and J. Fan, "Checking only when it is necessary: Enabling integrity auditing based on the keyword with sensitive information privacy for encrypted cloud data," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 6, pp. 3774–3789, Nov./Dec. 2022.
- [39] J. Li et al., "Searchable symmetric encryption with forward search privacy," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 1, pp. 460–474, Jan./Feb. 2021.
- [40] J. Li, T. Li, Z. Liu, and X. Chen, "Secure deduplication system with active key update and its application in IoT," *ACM Trans. Intell. Syst. Technol.*, vol. 10, no. 6, Oct. 2019. doi: [10.1145/3356468](https://doi.org/10.1145/3356468).
- [41] Z. Liu et al., "Encodeore: Reducing leakage and preserving practicality in order-revealing encryption," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 3, pp. 1579–1591, May/Jun. 2022.



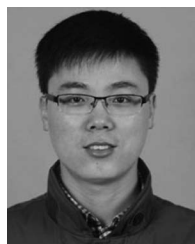
Ran Ding received the BEng and PhD degrees in computer science and technology from Anhui University, China. She is currently a lecturer in the School of Biomedical Engineering, Anhui Medical University. Her research interests cover privacy-preserving data sharing, cloud computing security, and applied cryptography.



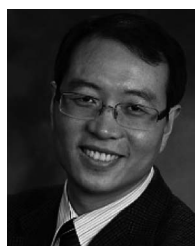
Yan Xu received the BS and MS degrees from Shandong University, Jinan, China, in 2004 and 2007, respectively, and the PhD degree from the University of Science and Technology of China, Hefei, China, in 2015. She is currently an associate professor with the School of Computer Science and Technology, Anhui University, Hefei. Her research interests include information security and applied cryptography.



Hong Zhong (Member, IEEE) received the PhD degree in computer science from the University of Science and Technology of China, in 2005. She is currently a professor and PhD Supervisor of the School of Computer Science and Technology with Anhui University. Her research interests include applied cryptography, vehicular ad hoc network, cloud computing security and software-defined networking (SDN). She has more than 120 scientific publications in reputable journals, academic books and international conferences.



Jie Cui received the PhD degree from the University of Science and Technology of China, in 2012. He is currently a professor and PhD supervisor with the School of Computer Science and Technology, Anhui University. His current research interests include applied cryptography, IoT security, vehicular ad hoc network, cloud computing security and software-defined networking (SDN). He has more than 100 scientific publications in reputable journals, academic books and international conferences.



Kewei Sha (Senior Member, IEEE) received the PhD degree in computer science from Wayne State University, in 2008. He is currently an associate professor of computer science and the associate director of the Cyber Security Institute, University of Houston-Clear Lake (UHCL). His research interests include the Internet of Things, cyber-physical systems, edge computing, security and privacy, and data management and analytics.